14. Experiment: Perform Transformation Using Homography Matrix

Aim

To perform a perspective transformation on an image using a Homography Matrix by mapping four points from the source image to their corresponding points in the destination image using OpenCV.

Algorithm

1. Start the program.
2. Import the required libraries (OpenCV, NumPy, and Matplotlib).
3. Read the input image.
4. Convert the image from BGR to RGB format for display.
5. Select four source points on the input image.
6. Define the corresponding four destination points.
7. Compute the Homography Matrix using cv2.findHomography().
8. Apply the perspective transformation using cv2.warpPerspective().
9. Display the original image and the transformed image.
10. Print the Homography Matrix.
11. Stop the program.

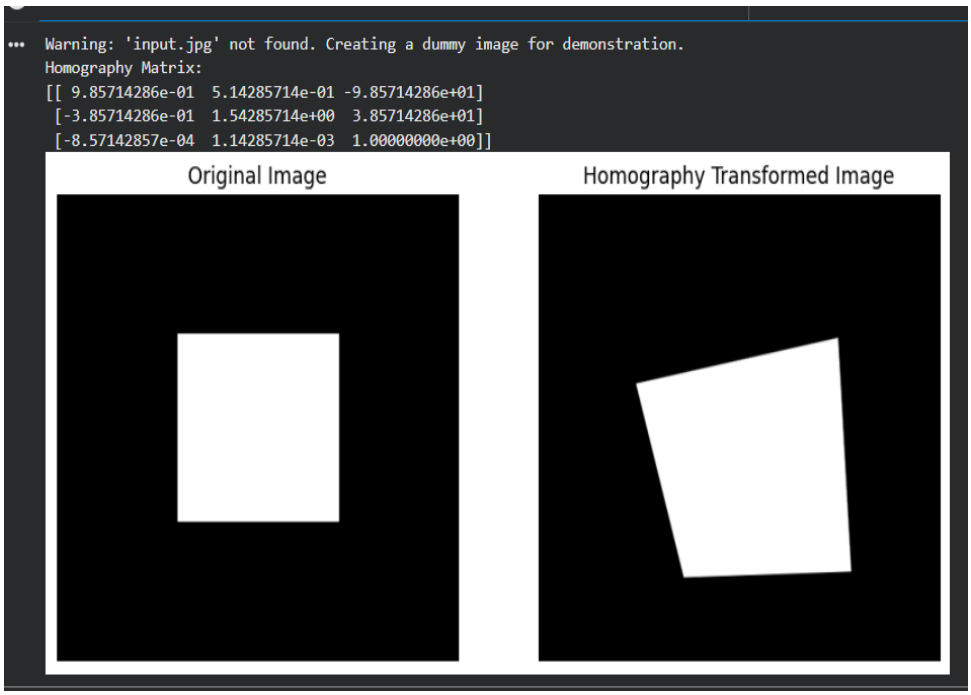
Program

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


# Load the input image

img = cv2.imread('input.jpg')


# Check if image was loaded successfully
```

```
if img is None:
    print("Warning: 'input.jpg' not found. Creating a dummy image for demonstration.")
    img = np.zeros((500, 500, 3), dtype=np.uint8)
    cv2.rectangle(img, (150, 150), (350, 350), (255, 255, 255), -1
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
src_pts = np.float32([
    [100, 100],
    [400, 100],
    [100, 400],
    [400, 400]
])
dst_pts = np.float32([
    [50, 150],
    [450, 50],
    [150, 450],
    [450, 450]
])
H, status = cv2.findHomography(src_pts, dst_pts)
print("Homography Matrix:")
print(H)
height, width = img.shape[:2]
warped = cv2.warpPerspective(img_rgb, H, (width, height))
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")
```

```
plt.subplot(1,2,2)
plt.imshow(warped)
plt.title("Homography Transformed Image")
plt.axis("off")
plt.show()
```

OUTPUT



Experiment 15: Perform Transformation using Direct Linear Transformation (DLT)

Aim

To perform image transformation using the Direct Linear Transformation (DLT) algorithm by computing the homography matrix from corresponding points and applying the perspective transformation to the input image.

Algorithm

1. Start the program.
2. Import the required libraries (OpenCV, NumPy, and Matplotlib).
3. Load the input image.
4. Define four corresponding source points and destination points.
5. Construct the DLT matrix Ausing the corresponding point pairs.
6. Compute the Singular Value Decomposition (SVD) of matrix A.
7. Obtain the homography matrix from the last row of the V^Tmatrix.
8. Normalize the homography matrix.
9. Apply the perspective transformation using cv2.warpPerspective().
10. Display the original image and the transformed image.
11. Print the computed homography matrix.
12. Stop the program.

Program

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread("input.jpg")

if img is None:
```

```
    print("Warning: 'input.jpg' not found. Creating a dummy image for demonstration.")
    img = np.zeros((500, 500, 3), dtype=np.uint8)
    cv2.rectangle(img, (150, 150), (350, 350), (255, 255, 255), -1
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
src_pts = np.array([
    [100, 100],
    [400, 100],
    [400, 400],
    [100, 400]
], dtype=np.float32)
dst_pts = np.array([
    [50, 150],
    [450, 100],
    [400, 450],
    [100, 400]
], dtype=np.float32)
A = []
for i in range(4):
    x, y = src_pts[i]
    u, v = dst_pts[i]
    A.append([-x, -y, -1, 0, 0, 0, u*x, u*y, u])
    A.append([0, 0, 0, -x, -y, -1, v*x, v*y, v])
A = np.array(A)
```

```
U, S, Vt = np.linalg.svd(A)

H = Vt[-1].reshape(3, 3)

H = H / H[2, 2]

print("Homography Matrix (DLT):")

print(H)

height, width = img.shape[:2]

warped = cv2.warpPerspective(img, H, (width, height))

plt.figure(figsize=(10,5))

plt.subplot(1,2,1)

plt.imshow(img)

plt.title("Original Image")

plt.axis("off")

plt.subplot(1,2,2)

plt.imshow(warped)

plt.title("DLT Transformed Image")

plt.axis("off")

plt.show()
```

output